웹 개발자 부트캠프 과정

CodingOn

HTTP Content-Type

참고. MIME Type

- Multipurpose Internet Main Extensions
- 인터넷에서 파일의 형식을 나타내는 표준
- 이메일에 첨부된 파일의 형식을 나타내기 위해 처음 개발되었으나 현재는 웹에서 광범위하게 사용됨

- text/plain, text/html
- image/jpeg, image/png
- audio/mpeg
- video/mp4
- application/pdf, application/vnd.ms-excel, application/json, application/zip

HTTP Content-Type

```
POST /api/users HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
Content-Type: application/json
Content-Length: 57
Accept: application/json

{
  "name": "Jane Doe",
  "email": "jane.doe@example.com"
}
```

요청 라인
요청 헤더
요청 바디

*빈 줄: 헤더와 본문 구분

- 요청 바디에 들어가는 타입을 요청 헤더의 Content-Type 에서 명시
- Content-Type 는 HTTP 통신에서 MIME Type 값을 전달하는 헤더

- 일반 텍스트 및 HTML
  - text/plain : 일반 텍스트
  - text/html : HTML 문서
- 폼 데이터
  - **application/x-www-form-urlencoded** : URL 인코딩된 폼 데이터 (기본 폼 제출 방식)
  - **multipart/form-data** : 파일 업로드와 같은 바이너리 데이터 포함 폼 데이터
- JSON 및 XML
  - **application/json** : JSON 형식의 데이터
  - application/xml : XML 형식의 데이터

application/x-www-form-urlencoded

- 주로 HTML 폼 데이터를 서버로 전송할 때 사용 **(일반 폼 전송)**
- 키와 값 쌍으로 이루어진 데이터를 URL 인코딩 방식으로 변환
  - ex. 공백은 "+"로, 특수 문자는 퍼센트 인코딩(%xx)으로 변환

POST

```
<form action="/submit" method="POST">
  <input type="text" name="name" value="John Doe">
  <input type="number" name="age" value="23">
  <input type="submit" value="Submit">
</form>
```

폼 제출 시 브라우저가
다음 형태로 서버에 전송

name=John+Doe&age=23

GET

```
<form action="/submit" method="GET">
  <input type="text" name="name" value="John Doe">
  <input type="number" name="age" value="23">
  <input type="submit" value="Submit">
</form>
```

폼 제출 시 브라우저가 데이터를
URL 쿼리 스트링 형식으로 인코딩

/submit?name=John+Doe&age=23

application/json

- JavaScript 라이브러리(예: axios, Fetch API)를 사용하여 클라이언트 측에서 **비동기적으로 데이터를 서버로 전송할 때** 주로 사용 **(동적 폼 전송)**
- 사용자가 웹 페이지에서 폼을 작성하고, JavaScript 코드가 이 데이터를 **JSON 형식**으로 서버에 전송

```html
<form id="myForm">
  <input type="text" name="name" value="John Doe">
  <input type="number" name="age" value="23">
  <button type="button" onclick="submitForm()">Submit</button>
</form>
<script>
  function submitForm() {
    const formData = {
      name: document.querySelector('input[name="name"]').value,
      age: document.querySelector('input[name="age"]').value
    };

    axios.post('/submit', formData, {
      headers: {
        'Content-Type': 'application/json'
      }
    })
      .then(response => {
        console.log(response.data);
      })
      .catch(error => {
        console.error('There was an error!', error);
      });
  }
</script>
```

폼 제출 시 JavaScript 코드가 데이터를 JSON 형식으로 서버에 전송

```
{
  "name": "John Doe",
  "age": 23
}
```

multipart/form-data

- **파일 업로드**를 포함한 복잡한 폼 데이터 전송에 사용

```html
<form method="POST" action="/submit" enctype="multipart/form-data">
    <input type="text" name="username" value="john_doe">
    <input type="file" name="profile_picture">
    <input type="submit" value="Submit">
</form>
```

```
--boundary
Content-Disposition: form-data; name="username"

john_doe
--boundary
Content-Disposition: form-data; name="profile_picture"; filename="photo.jpg"
Content-Type: image/jpeg

(바이너리 데이터)
--boundary--
```

클라이언트에서 서버로 요청 시
전송되는 데이터 형태

API (Get, Post)

GET Method

- Get Method URL 매핑 시 Controller 에서 **@GetMapping(url)**을 이용

```
@GetMapping(url주소)
public String 함수이름() {
    return 템플릿 파일 명;
}
```

GET Method. query string

- URL query string (?key=value) 을 Controller 에서 **@RequestParam** 이용해 받아오기

```
@GetMapping(url주소)
public String 함수이름(@RequestParam(value="key") String key) {
  return 템플릿 파일 명;
}
```

- 메서드의 매개변수 key 에 query string 의 value 값이 전달됨
- @RequestParam에 required=false 값을 설정하면 없어도 실행됨

GET Method. path variable

- URL path variable 을 Controller에서 **@PathVariable** 이용하여 받아오기

```
@GetMapping(url주소/{value})
public String 함수이름(@PathVariable String value) {
  return 템플릿 파일 명;
}
```

- 메서드의 매개변수 value 에 URL path variable 로 입력된 값이 전달됨

GET Method. path variable

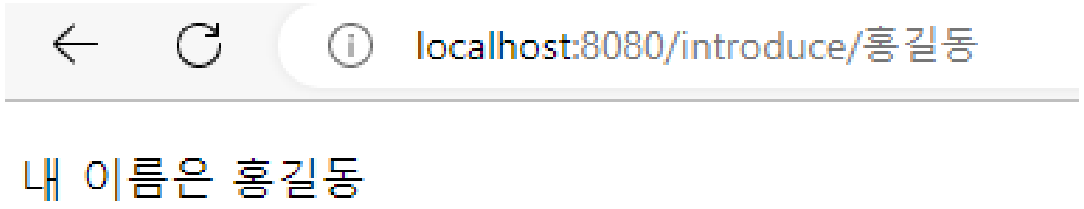
- 만약, path variable 을 다르게 설정하고 싶다면?

```
@GetMapping(url주소/{value}/{value2})
public String 함수이름(@PathVariable String value, @PathVariable("value2") String abc) {
  return 템플릿 파일 명;
}
```

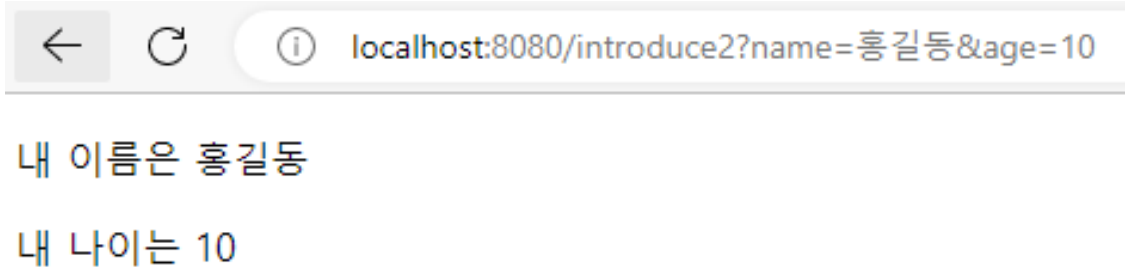
- @Pathvariable의 괄호 안에 경로변수를 작성하고 원하는 메서드의 매개변수명 사용
  - 위 예제의 경우 value2 경로 변수의 값이 abc 라는 변수에 저장
  - ex) GET “URL주소/1/2” 라는 요청 시, abc 변수에 2가 저장됨

실습. GET 실습

1. "http://localhost:8080/introduce/홍길동" 으로 접속했을 때 아래와 같이 나오게 만들기



2. "http://localhost:8080/introduce2?name=홍길동&age=10"으로 접속했을 때 아래와 같이 나온다 할 때, 본인 이름으로 변경해서 실습



POST Method

- Post Method URL 매핑 시 Controller 에서 **@PostMapping(url)**을 이용

```
@PostMapping(url주소)
public String 함수이름() {
    return 템플릿 파일 명;
}
```

POST Method

```html
// API-post.html
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <title>Title</title>
</head>
<body>
  <form action="mvc-post" method="post">
    <input type="text" name="name">
    <input type="number" name="age">
    <button>전송</button>
  </form>
</body>
</html>
```

```java
@PostMapping("mvc-post")
public String postMVC(
        @RequestParam(value = "name", required = false) String name,
        @RequestParam(value = "age", required = false) String age,
        Model model) {
    model.addAttribute("name", name);
    model.addAttribute("age", age);
    return "API";
}
```

API로 이용하기

- 앞에서 배운 방식은 return을 이용해 Template view를 무조건 불러왔다.
- API로만, 데이터만 전달하고 싶을 때는?
- @ResponseBody 이용하기

```
@GetMapping("response-string")
@ResponseBody
public String getString(@RequestParam("name") String name) { return "hello " + name; }
```

- express 에서의 res.send() 와 유사

실습. POST 실습

- 이름, 성별, 생년월일, 관심사 를 포함하여 4개 이상의 정보를 입력 받고, POST 요청으로 Form 전송하기

폼 전송 - 실습

이름
성별 ○남자 ○여자
생년월일 1950 년 1 월 1 일
관심사 ☐여행 ☐패션 ☐음식

회원가입

이름 : 홍길동
성별 : 남자
생년월일 : 1976-5-15
관심사 :여행,패션,음식

DTO 이용하기

```html
<h1>DTO 이용</h1>
<div class="forms-container">
 <form action="/dto/res1" method="GET" class="form-card">
  <h3>#8. DTO with GET Method</h3>
  <input type="text" name="name" placeholder="이름" /><br />
  <input type="number" name="age" placeholder="나이" /><br />
  <button type="submit">GET 전송</button>
 </form>
 <form action="/dto/res2" method="POST" class="form-card">
  <h3>#9. DTO with POST Method (@RequestBody x)</h3>
  <input type="text" name="name" placeholder="이름" /><br />
  <input type="number" name="age" placeholder="나이" /><br />
  <button type="submit">Post 전송</button>
 </form>
 <form action="/dto/res3" method="POST" class="form-card">
  <h3>#10. DTO with POST Method (@RequestBody o)</h3>
  <input type="text" name="name" placeholder="이름" /><br />
  <input type="number" name="age" placeholder="나이" /><br />
  <button type="submit">Post 전송</button>
 </form>
</div>
```

```java
public class UserDTO {
   private int age;
   private String name;

   public int getAge() {
      return age;
   }

   public void setAge(int age) {
      this.age = age;
   }

   public String getName() {
      return name;
   }

   public void setName(String name) {
      this.name = name;
   }
}
```

실습. DTO 이용하기 – 일반 폼

```java
@GetMapping("/dto/res1")
@ResponseBody
public String dtoRes1(@ModelAttribute UserDTO userDTO) {
    System.out.println("[GET] userDTO (name) = " + userDTO.getName());
    System.out.println("[GET] userDTO (age) = " + userDTO.getAge());

    return userDTO.getName() + " " + userDTO.getAge();
}

@PostMapping("/dto/res2")
@ResponseBody
public String dtoRes2(UserDTO userDTO) {
    System.out.println("[POST] userDTO (name) = " + userDTO.getName());
    System.out.println("[POST] userDTO (age) = " + userDTO.getAge());

    return userDTO.getName() + " " + userDTO.getAge();
}

@PostMapping("/dto/res3")
@ResponseBody
public String dtoRes22(@RequestBody UserDTO userDTO) {
    System.out.println("[POST] userDTO (name) = " + userDTO.getName());
    System.out.println("[POST] userDTO (age) = " + userDTO.getAge());

    return userDTO.getName() + " " + userDTO.getAge();
}
```

Controller 코드를 다음과 같이 작성하고 프론트 요청 시 어떻게 return 되는지 확인하기

Slack #과제 채널에 return 여부를 제출!

- GET /dto/res1: O/X
- POST /dto/res2: O/X
- POST /dto/res3: O/X

VO 이용하기

```html
<h1>VO 이용</h1>
<div class="forms-container">
  <form action="/vo/res1" method="GET" class="form-card">
    <h3>#11. VO with GET Method</h3>
    <input type="text" name="name" placeholder="이름" /><br />
    <input type="text" name="age" placeholder="나이" /><br />
    <button type="submit">GET 전송</button>
  </form>
  <form action="/vo/res2" method="POST" class="form-card">
    <h3>#12. VO with POST Method (@RequestBody x)</h3>
    <input type="text" name="name" placeholder="이름" /><br />
    <input type="text" name="age" placeholder="나이" /><br />
    <button type="submit">Post 전송</button>
  </form>
  <form action="/vo/res3" method="POST" class="form-card">
    <h3>#13. VO with POST Method (@RequestBody o)</h3>
    <input type="text" name="name" placeholder="이름" /><br />
    <input type="text" name="age" placeholder="나이" /><br />
    <button type="submit">Post 전송</button>
  </form>
</div>
```

```java
public class UserVO {
    private String name;
    private int age;

    public String getName() {
        return name;
    }

    public int getAge() {
```

실습. VO 이용하기 – 일반 폼

```
@GetMapping("/vo/res1")
@ResponseBody
public String voRes1(@ModelAttribute UserVO userVO) {
    System.out.println("[GET] userVO (name) = " + userVO.getName());
    System.out.println("[GET] userVO (age) = " + userVO.getAge());

    return userVO.getName() + " " + userVO.getAge();
}

@PostMapping("/vo/res2")
@ResponseBody
public String voRes2(UserVO userVO) {
    System.out.println("[POST] userVO (name) = " + userVO.getName());
    System.out.println("[POST] userVO (age) = " + userVO.getAge());

    return userVO.getName() + " " + userVO.getAge();
}

@PostMapping("/vo/res3")
@ResponseBody
public String voRes3(@RequestBody UserVO userVO) {
    System.out.println("[POST] userVO (name) = " + userVO.getName());
    System.out.println("[POST] userVO (age) = " + userVO.getAge());

    return userVO.getName() + " " + userVO.getAge();
}
```

Controller 코드를 다음과 같이 작성하고
프론트 요청 시 어떻게 return 되는지 확인하기

Slack #과제 채널에 return 여부를 제출!

- GET /vo/res1: O/X
- POST /vo/res2: O/X
- POST /vo/res3: O/X

Axios 이용하기

Axios with. DTO – HTML 코드

```
<h1>DTO 이용 with. Axios</h1>
<div class="box">
 <div id="dto-result-container" class="result-container"></div>
 <div class="forms-container">
  <form id="form_dto1" class="form-card">
   <h3>#14. DTO with Get Method, Axios</h3>
   <input type="text" name="name" placeholder="이름" /><br />
   <input type="text" name="age" placeholder="나이" /><br />
   <button type="button" onclick="dtoRes1();">GET (일반) - axios</button>
   <button type="button" onclick="dtoRes2();">Get (DTO) - axios</button>
  </form>
  <form id="form_dto2" class="form-card">
   <h3>#15. DTO with POST Method, Axios</h3>
   <input type="text" name="name" placeholder="이름" /><br />
   <input type="text" name="age" placeholder="나이" /><br />
   <button type="button" onclick="dtoRes3();">
    Post (일반) - axios
   </button>
   <button type="button" onclick="dtoRes4();">Post (DTO) - axios</button>
   <button type="button" onclick="dtoRes5();">Post (DTO) - axios</button>
  </form>
 </div>
</div>
```

Axios with. DTO – JavaScript 코드

```javascript
const dtoResultBox = document.getElementById('dto-result-container');

// 폼 데이터를 쿼리 문자열로 만들어 GET /axios/res1 요청
function dtoRes1() {
  const form = document.getElementById('form_dto1');
  axios
    .get(`/axios/res1?name=${form.name.value}&age=${form.age.value}`)
    .then((res) => {
      console.log('dtoRes1: ', res.data);
      dtoResultBox.textContent = res.data;
    });
}

// 폼 데이터를 쿼리 문자열로 만들어 GET /axios/res2 요청
function dtoRes2() {
  const form = document.getElementById('form_dto1');
  axios
    .get(`/axios/res2?name=${form.name.value}&age=${form.age.value}`)
    .then((res) => {
      console.log('dtoRes2: ', res.data);
      dtoResultBox.textContent = res.data;
    });
}
```

```javascript
function dtoRes3() {
  const form = document.getElementById('form_dto2');
  axios
    .post(`/axios/res3`, { name: form.name.value, age: form.age.value })
    .then((res) => {
      console.log('dtoRes3: ', res.data);
    })
    .catch((error) => {
      console.error('Error in dtoRes3:', error);
      dtoResultBox.textContent = error.message;
    });
}

function dtoRes4() {
  const form = document.getElementById('form_dto2');
  axios
    .post(`/axios/res4`, { name: form.name.value, age: form.age.value })
    .then((res) => {
      console.log('dtoRes1: ', res.data);
      dtoResultBox.textContent = res.data;
    });
}

function dtoRes5() {
  const form = document.getElementById('form_dto2');
  axios
    .post(`/axios/res5`, { name: form.name.value, age: form.age.value })
    .then((res) => {
      console.log('dtoRes5: ', res.data);
      dtoResultBox.textContent = res.data;
    });
}
```

실습. Axios - DTO

Controller 코드를 다음과 같이 작성하고
프론트 요청 시 어떻게 return 되는지 확인하기

Slack #과제 채널에 return 여부를 제출!

- GET /axios/res1: O/X
- GET /axios/res2: O/X
- POST /axios/res3: O/X
- POST /axios/res4: O/X
- POST /axios/res5: O/X

```java
@GetMapping("/axios/res1")
@ResponseBody
public String axiosRes1(@RequestParam String name, @RequestParam String age){
   System.out.println("[GET] axios (name) = " + name);
   System.out.println("[GET] axios (age) = " + age);

   return "이름: " + name + ", 나이: " + age;
}

@GetMapping("/axios/res2")
@ResponseBody
public String axiosRes2(UserDTO userDTO){
   System.out.println("[GET] axios and dto (name) = " + userDTO.getName());
   System.out.println("[GET] axios and dto (age) = " + userDTO.getAge());

   return "이름: " + userDTO.getName() + ", 나이: "+ userDTO.getAge();
}

@PostMapping("/axios/res3")
@ResponseBody
public String axiosRes3(@RequestParam String name, @RequestParam String age) {
   System.out.println("[POST] axios (name) = " + name);
   System.out.println("[POST] axios (age) = " + age);

   return "이름: " + name + ", 나이: " + age;
}

@PostMapping("/axios/res4")
@ResponseBody
public String axiosRes4(UserDTO userDTO){
   System.out.println("[POST] axios and dto (name) = " + userDTO.getName());
   System.out.println("[POST] axios and dto (age) = " + userDTO.getAge());

   return "이름:" + userDTO.getName() + ", 나이: "+ userDTO.getAge();
}

@PostMapping("/axios/res5")
@ResponseBody
public String axiosRes5(@RequestBody UserDTO userDTO){
   System.out.println("[POST] axios and dto (name) = " + userDTO.getName());
   System.out.println("[POST] axios and dto (age) = " + userDTO.getAge());

   return "이름:" + userDTO.getName() + ", 나이: "+ userDTO.getAge();
}
```

Axios with. VO – HTML 코드

```
<h1>VO 이용 with. Axios</h1>
<div class="box">
 <div id="vo-result-container" class="result-container"></div>
 <div class="forms-container">
  <form id="form_vo1" class="form-card">
   <h3>#16. VO with GET Method, Axios</h3>
   <input type="text" name="name" placeholder="이름" /><br />
   <input type="text" name="age" placeholder="나이" /><br />
   <button type="button" onclick="voRes1();">GET (일반) - axios</button>
   <button type="button" onclick="voRes2();">Get (VO) - axios</button>
  </form>
  <form id="form_vo2" class="form-card">
   <h3>#17. VO with POST Method, Axios</h3>
   <input type="text" name="name" placeholder="이름" /><br />
   <input type="text" name="age" placeholder="나이" /><br />
   <button type="button" onclick="voRes3();">Post (일반) - axios</button>
   <button type="button" onclick="voRes4();">Post (VO) - axios</button>
   <button type="button" onclick="voRes5();">Post (VO) - axios</button>
  </form>
 </div>
</div>
```

Axios with. VO – JavaScript 코드

```javascript
const voResultBox = document.getElementById('vo-result-container');

function voRes1() {
  const form = document.getElementById('form_vo1');
  axios
    .get(`/axios/vo/res1?name=${form.name.value}&age=${form.age.value}`)
    .then((res) => {
      console.log('voRes1: ', res.data);
      voResultBox.textContent = res.data;
    });
}

function voRes2() {
  const form = document.getElementById('form_vo1');
  axios
    .get(`/axios/vo/res2?name=${form.name.value}&age=${form.age.value}`)
    .then((res) => {
      console.log('voRes2: ', res.data);
      voResultBox.textContent = res.data;
    });
}
```

```javascript
function voRes3() {
  const form = document.getElementById('form_vo2');
  axios
    .post(`/axios/vo/res3`, { name: form.name.value, age: form.age.value })
    .then((res) => {
      console.log('voRes3: ', res.data);
    })
    .catch((error) => {
      console.error('Error in voRes3:', error);
      voResultBox.textContent = error.message;
    });
}

function voRes4() {
  const form = document.getElementById('form_vo2');
  axios
    .post(`/axios/vo/res4`, { name: form.name.value, age: form.age.value })
    .then((res) => {
      console.log('voRes4: ', res.data);
      voResultBox.textContent = res.data;
    });
}

function voRes5() {
  const form = document.getElementById('form_vo2');
  axios
    .post(`/axios/vo/res5`, { name: form.name.value, age: form.age.value })
    .then((res) => {
      console.log('voRes5 : ', res.data);
      voResultBox.textContent = res.data;
    });
}
```

실습. Axios - VO

Controller 코드를 다음과 같이 작성하고
프론트 요청 시 어떻게 return 되는지 확인하기

Slack #과제 채널에 return 여부를 제출!

- GET /axios/vo/res1: O/X
- GET /axios/vo/res2: O/X
- POST /axios/vo/res3: O/X
- POST /axios/vo/res4: O/X
- POST /axios/vo/res5: O/X

```java
@GetMapping("/axios/vo/res1")
@ResponseBody
public String axiosVoRes1(@RequestParam String name, @RequestParam String age) {
    System.out.println("[GET] axios (name) = " + name);
    System.out.println("[GET] axios (age) = " + age);

    return "이름: " + name + ", 나이: " + age;
}

@GetMapping("/axios/vo/res2")
@ResponseBody
public String axiosVoRes2(UserVO userVO) {
    System.out.println("[GET] axios and vo (name) = " + userVO.getName());
    System.out.println("[GET] axios and vo (age) = " + userVO.getAge());

    return "이름: "+ userVO.getName() + ", 나이: "+ userVO.getAge();
}

@PostMapping("/axios/vo/res3")
@ResponseBody
public String axiosVoRes3(@RequestParam String name, @RequestParam String age) {
    System.out.println("[POST] axios (name) = " + name);
    System.out.println("[POST] axios (age) = " + age);

    return "이름: " + name + ", 나이: " + age;
}

@PostMapping("/axios/vo/res4")
@ResponseBody
public String axiosVoRes4(UserVO userVO){
    System.out.println("[POST] axios and vo (name) = " + userVO.getName());
    System.out.println("[POST] axios and vo (age) = " + userVO.getAge());

    return "이름: " + userVO.getName() + ", 나이: " + userVO.getAge();
}

@PostMapping("/axios/vo/res5")
@ResponseBody
public String axiosVoRes5(@RequestBody UserVO userVO){
    System.out.println("[POST] axios and vo (name) = " + userVO.getName());
    System.out.println("[POST] axios and vo (age) = " + userVO.getAge());

    return "이름: " + userVO.getName() + ", 나이: " + userVO.getAge();
}
```

정리

주요 어노테이션 정리 (1)

- **@Controller**
  - 해당 클래스가 Spring MVC 컨트롤러임을 나타냄
  - 주로 뷰를 반환하는 웹 요청을 처리
- **@GetMapping / @PostMapping**
  - HTTP GET/POST 요청을 특정 핸들러 메서드에 매핑
  - URL 경로를 지정하여 해당 요청을 처리할 메서드를 정의
- **@ModelAttribute**
  - HTTP 요청 파라미터를 객체에 바인딩
  - 주로 폼 데이터를 객체로 변환할 때 사용
  - setter 메서드를 통해 값을 설정

주요 어노테이션 정리 (2)

- **@RequestParam**
  - Spring MVC 에서 HTTP 요청 파라미터를 메서드의 파라미터로 바인딩할 때 사용하는 어노테이션
  - ex. GET 요청의 쿼리 스트링 파라미터 처리
  - ex. POST 요청의 폼 데이터 처리

```
@GetMapping("/users")
public List<User> getUsers(@RequestParam String name) {
  // 이 경우 /users?name=John과 같은 요청에서 "John"을 name 변수로 받음
}
```

- **@PathVariable**
  - Spring MVC 에서 URI path variable 을 핸들러 메서드의 파라미터로 바인딩할 때 사용하는 어노테이션

```
@GetMapping("/users/{id}")
public User getUser(@PathVariable Long id) {
  // 메서드 구현
}
```

주요 어노테이션 정리 (3)

- **@ResponseBody**
  - Spring MVC 에서 컨트롤러 메서드의 반환 값을 HTTP 응답 본문(body)에 넣도록 지시하는 어노테이션
  - 주로 JSON, XML 같은 데이터 반환할 때 사용됨 (기본적으로 객체는 JSON 으로 변환되지만 다른 형식으로 변환 가능)
  - RESTful 웹 서비스 구현이 용이
  - 데이터를 직접 반환하므로 View 로직이 필요 없어짐 (프론트엔드와 백엔드가 분리된 아키텍처에서 유용)
  - 클라이언트 사이드 렌더링 (ex. SPA) 에 적합

```
@GetMapping("/user/{id}")
@ResponseBody
public User getUser(@PathVariable Long id) {
    return userService.findById(id);
}
```

- **@RestController**
  - 핸들러 메서드 위에 매번 @ResponseBody 를 붙이는 것은 번거로움
  - @RestController 어노테이션을 사용하면 클래스의 모든 메서드에 @ResponseBody가 적용됨

주요 결과 분석 (1)

- **일반 폼 전송**
  - @RequestParam: GET, POST 가능
  - @PathVariable: GET 가능
- **DTO 이용 일반 폼 전송**
  - GET 전송: 가능
  - POST 전송: @RequestBody 없으면 가능
  - POST 전송: @RequestBody 있으면 실패 (*Content-Type 불일치)
- **VO 이용 일반 폼 전송**
  - GET 전송: null (*setter 메서드 부재)
  - POST 전송: @RequestBody 없으면 null (*setter 메서드 부재)
  - POST 전송: @RequestBody 있으면 실패 (*Content-Type 불일치)

주요 결과 분석 (2)

- **DTO 이용 동적 폼 전송**
  - GET 일반: 가능
  - GET DTO: 가능
  - POST 일반: 실패 (*Content-Type 불일치)
  - POST DTO: @RequestBody 없으면 null
  - POST DTO: @RequestBody 있으면 가능

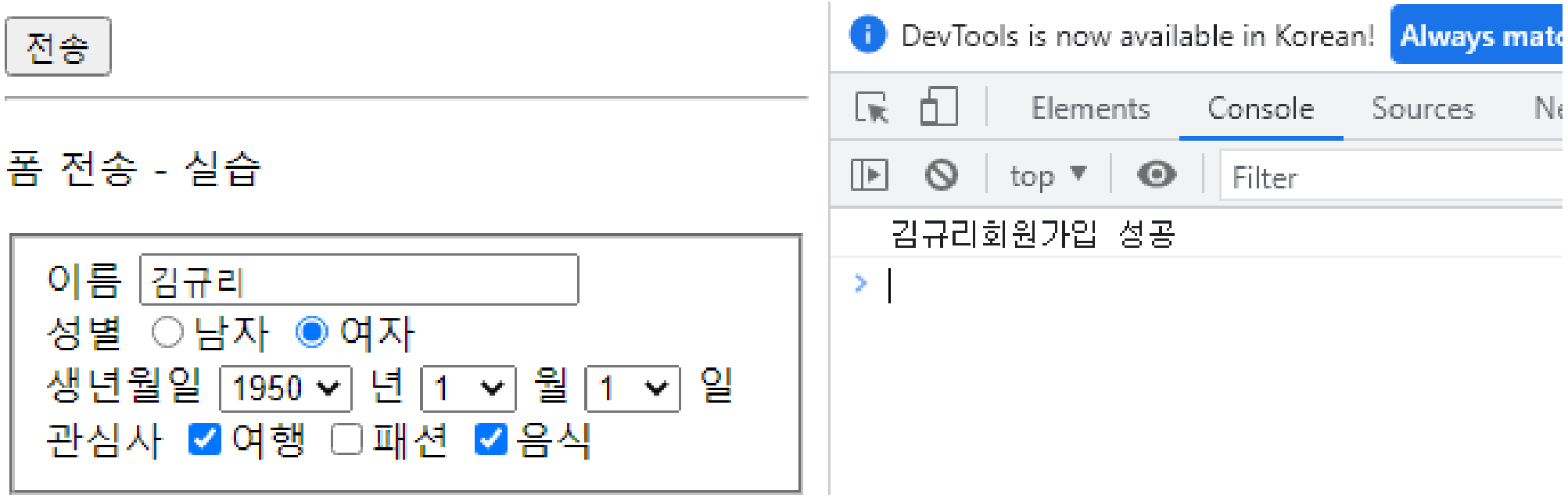
- **VO 이용 동적 폼 전송**
  - GET 일반: 가능
  - GET VO: null
  - POST 일반: 실패 (*Content-Type 불일치)
  - POST VO: @RequestBody 없으면 null
  - POST VO: @RequestBody 있으면 가능

주요 결과 분석 (3)

- DTO 사용 시: setter 메소드가 있어 일반 폼 전송 시 데이터 바인딩 잘 됨
- VO 사용 시: setter 메소드가 없어 일반 폼 전송 시 null/0 반환
- Axios 사용 시: JSON 데이터 전송 시 @RequestBody 사용하면 DTO/VO 모두 정상 동작

실습. 동적폼전송 실습

- Axios를 이용해 동적폼전송 구현하기 (VO 이용)



실습. 회원관리 시스템

- C : 회원 가입
- R : 로그인
- U : 회원 정보 수정
- D : 회원 탈퇴

- 위의 기능을 수행할 수 있는 RESTful 한 시스템을 만들어보기